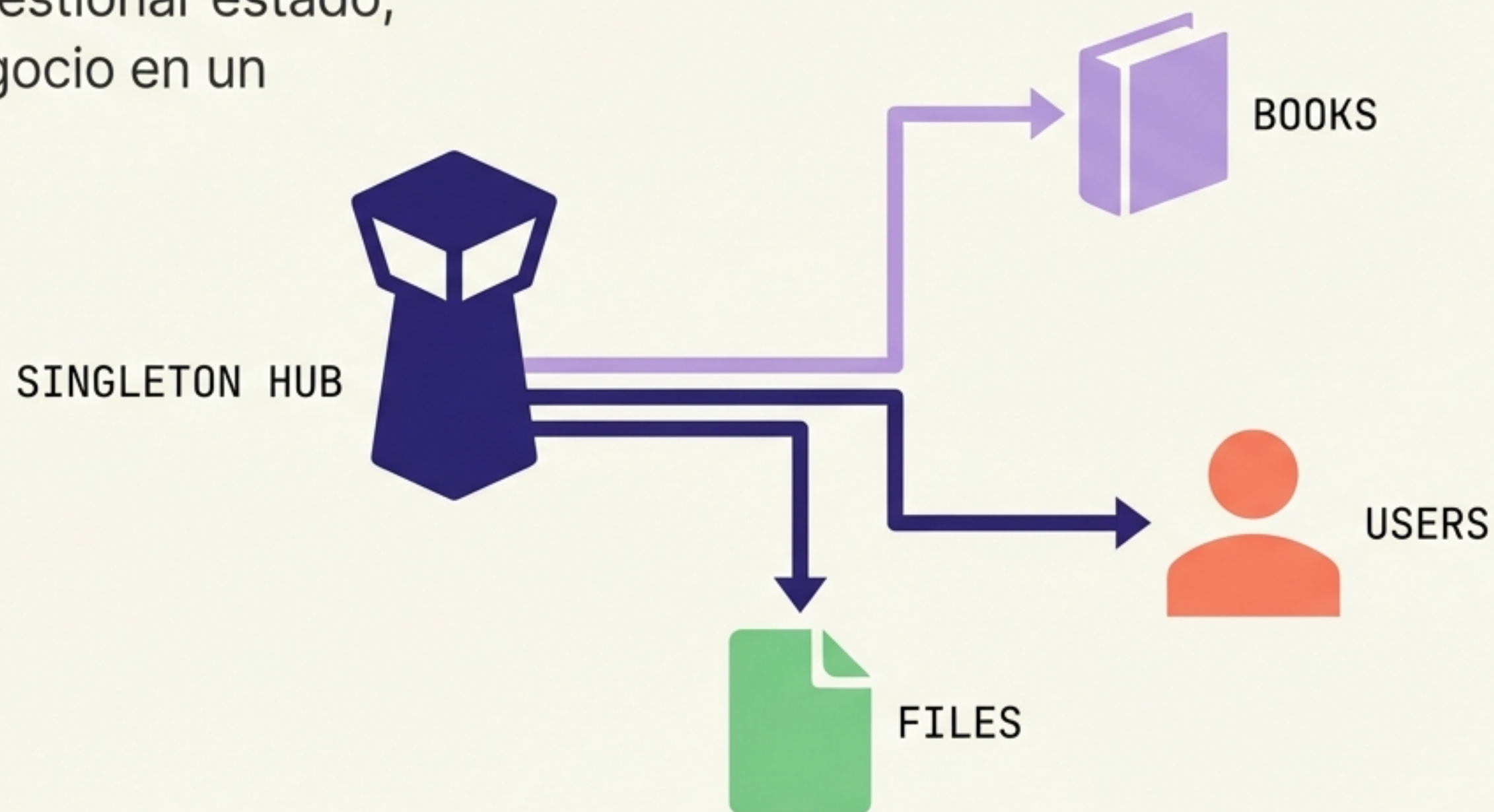


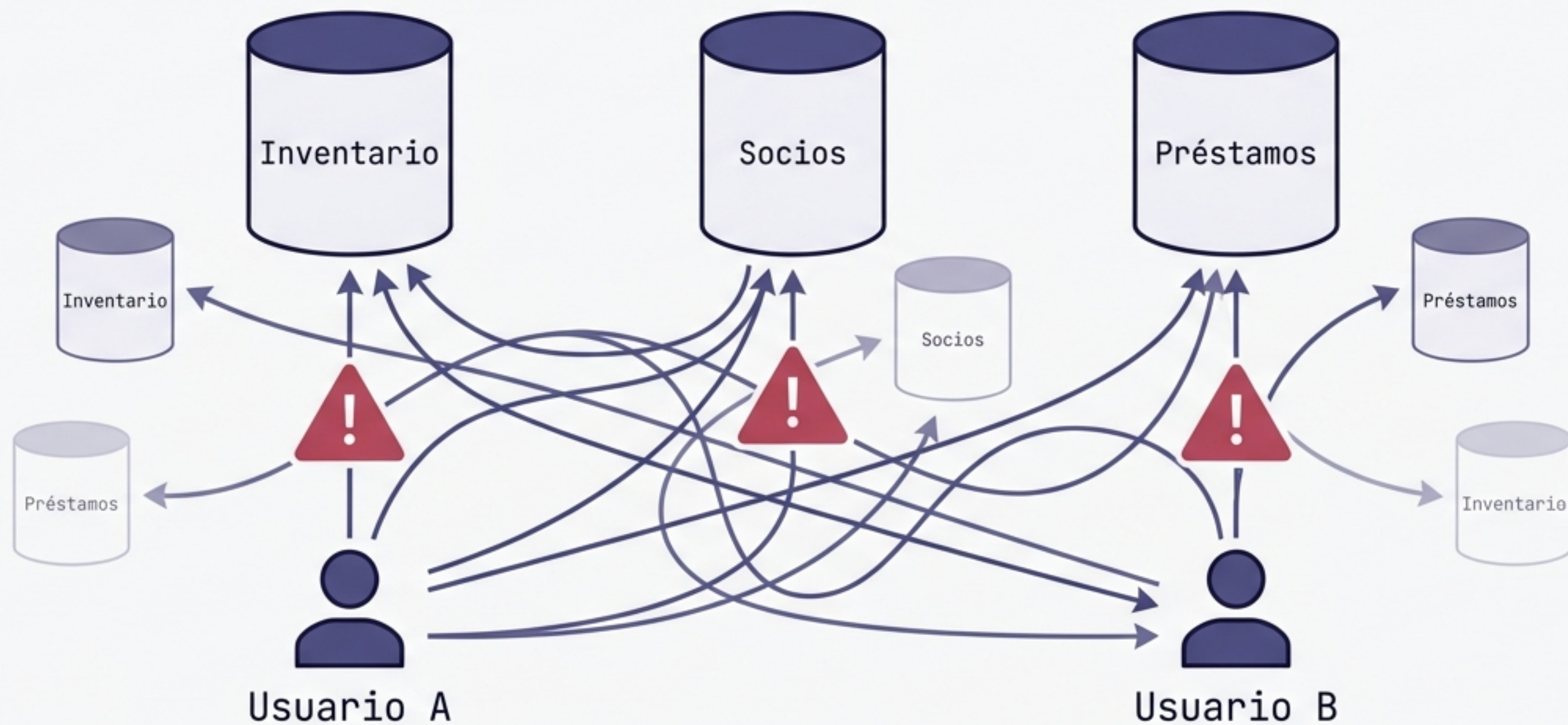
Arquitectura Avanzada con el Patrón Singleton en Kotlin

Un enfoque práctico para gestionar estado, persistencia y lógica de negocio en un sistema de biblioteca.



Múltiples instancias generan caos en tus datos

¿Cómo garantizamos que todos en el programa vean exactamente los mismos datos?
Instanciar múltiples gestores provoca **pérdida de información** y **desincronización**.

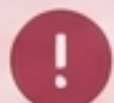


La palabra reservada object garantiza una única fuente

Kotlin elimina la necesidad de patrones complejos. Usar object asegura que solo exista un gestor en todo el programa.



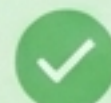
```
1 class Biblioteca { ... }  
2  
3 val b1 = Biblioteca()  
4 val b2 = Biblioteca()  
5
```



Múltiples instancias = Confusión



```
1 object Biblioteca { ... }  
2 }  
3  
4 Biblioteca.metodo()  
5
```



Instancia única = Control absoluto

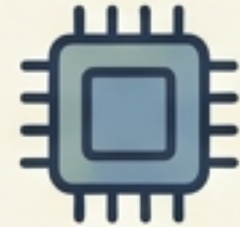
El Singleton protege y centraliza los datos en memoria



El estado está protegido del exterior. Nadie puede modificar las listas directamente.



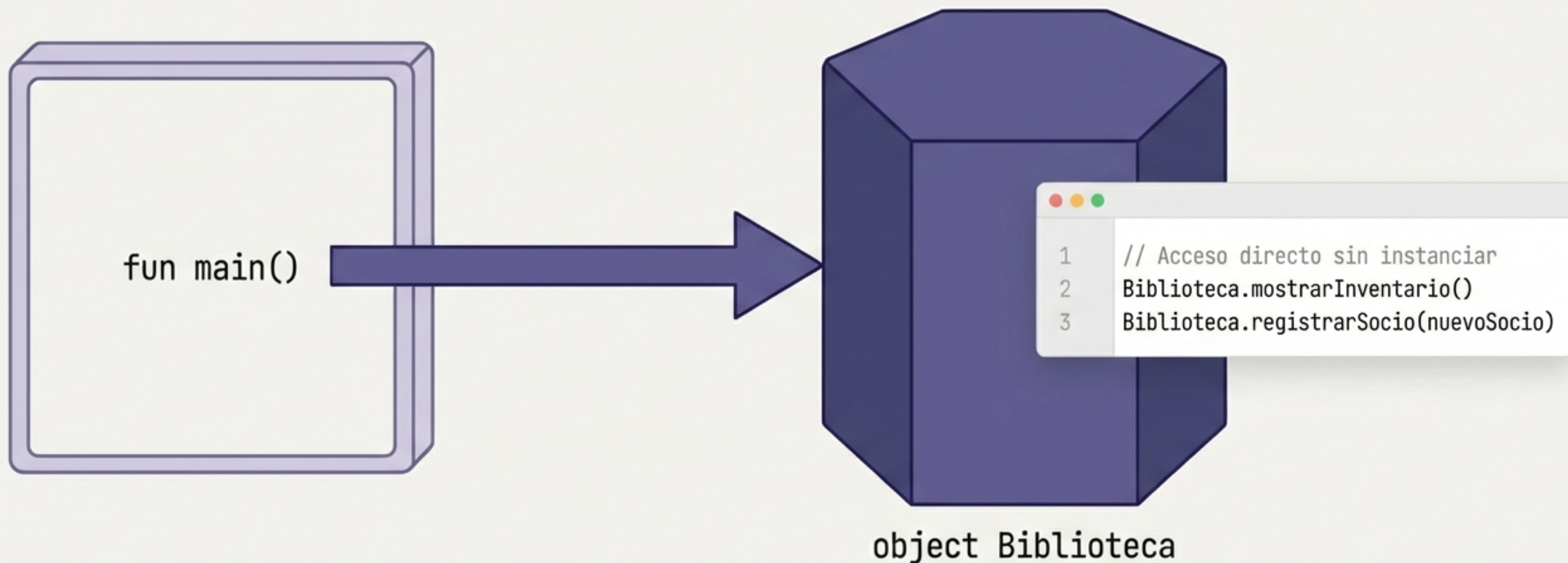
```
object Biblioteca {  
  ...  
  private val inventario ...  
  private val socios ...  
  private val prestamos ...  
  ...  
}
```



Almacena los datos temporalmente mientras el programa está en ejecución.

Cualquier parte del programa puede comunicarse con el gestor

No es necesario pasar la variable de la biblioteca de función en función.
El acceso es global, directo y seguro.



El gestor verifica estrictas reglas de negocio antes de actuar

El gestor no es solo un almacén; coordina la validación antes de autorizar cualquier cambio de estado.

```
object Biblioteca
```

```
  val socioEncontrado = ...
```

```
  Biblioteca.mostrarInventari(socioEncontrado != null)
```

```
  materialEncontrado = ...
```

```
  (materialEncontrado = materialEncontrado())
```

```
  // sotos.registrar(socioEncontrado, materialEncontrado)
```

```
  while (!Hay stock)
```

```
    (materialEncontrado.prestar())
```

```
    préstamo
```

```
  }
```

```
  return null
```

```
}
```

Paso 1: ¿Existe el socio? (socioEncontrado != null)

Paso 2: ¿Existe el material? (materialEncontrado != null)

Paso 3: ¿Ya tiene una copia? (Evitar duplicados)

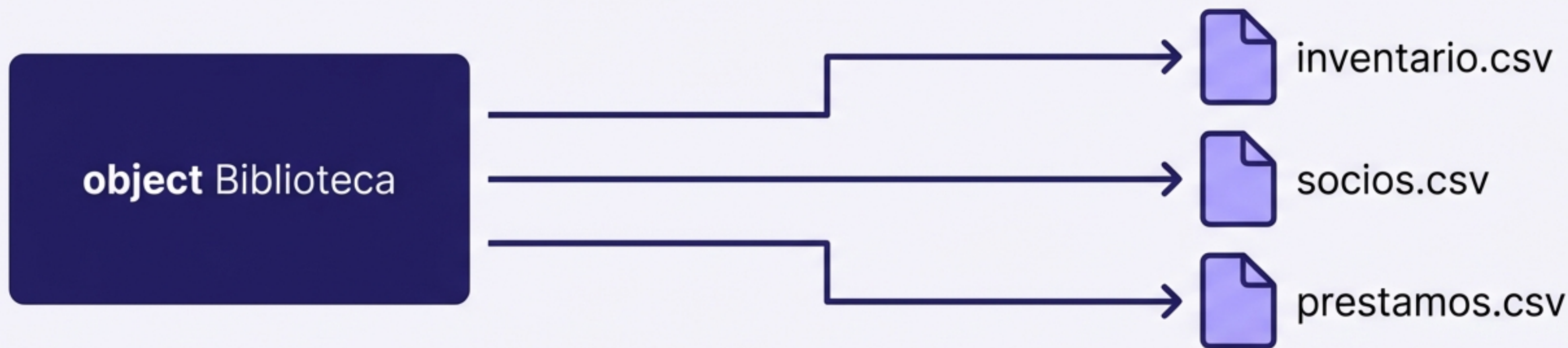
Paso 4: ¿Hay stock disponible? (materialEncontrado.prestar())



Préstamo Aprobado

Persistencia de estado mediante exportación a archivos CSV

Cuando el programa se cierra, la memoria se borra. El Singleton asume la responsabilidad de escribir su estado actual en el disco duro usando **java.io.File**.



```
// Dentro de guardarDatos()  
File("inventario.csv").writeText(textoInventario)  
File("socios.csv").writeText(textoSocios)
```


Reconstruyendo la memoria al iniciar el programa

El Singleton lee línea por línea con `readLines()`, divide los datos y resucita los objetos en sus listas privadas.

Libro;LIB-001;El
Quijote;Cervantes;1605...

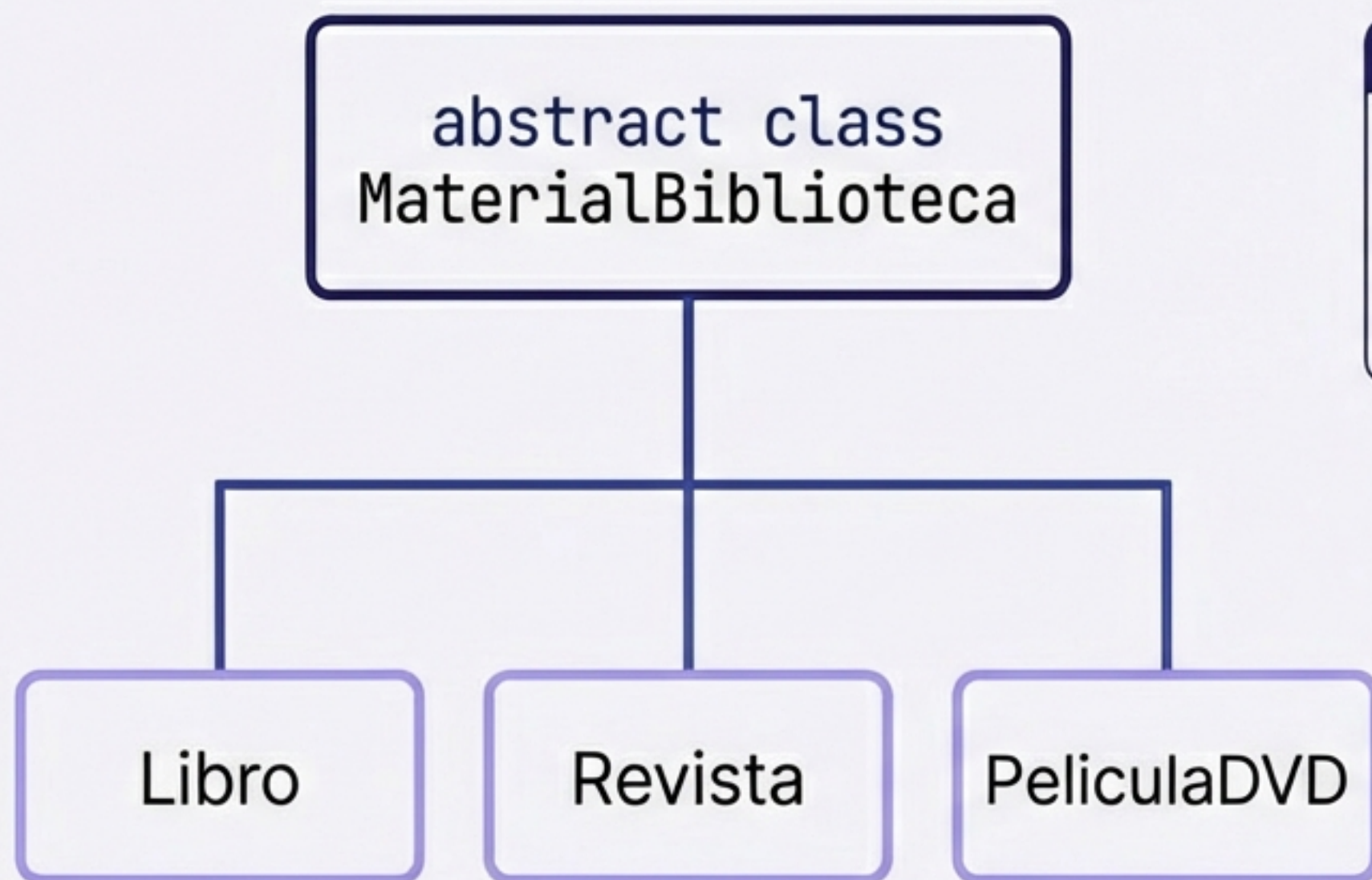
`split(";")`

Memoria del
Singleton

```
// Dentro de cargarDatos()
val datos = linea.split(";")
when (datos[0]) {
    "Libro" -> { material = Libro(...) }
}
```


Delegando responsabilidades con clases abstractas y herencia

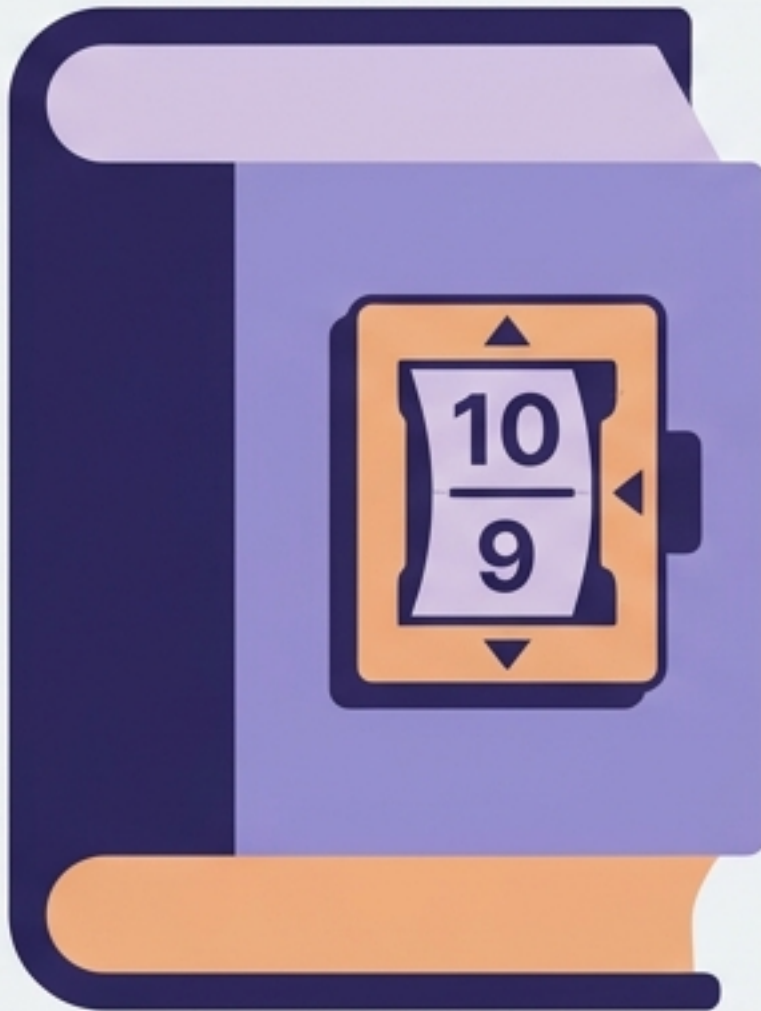
El Singleton maneja "Materiales". Gracias al polimorfismo, no le importa si son libros o DVDs; trata a todos por igual mientras las clases hijas definen sus propios detalles.



```
// Guardando el tipo dinámicamente  
return "${this::class.  
simpleName};$id;$titulo..."
```


Las entidades individuales gestionan su propio estado interno

El control de stock no ocurre en el Singleton. Cada material gestiona sus propias copias (copiasDisponibles). El Singleton simplemente les da la orden.



```
// Dentro de MaterialBiblioteca
fun prestar(): Boolean {
    if (copiasDisponibles > 0) {
        copiasDisponibles--
        return true
    }
    return false
}
```


Expresiones regulares aseguran la integridad de los datos

Funciones auxiliares como `pedirIdValido()` actúan como filtros estrictos antes de que la información llegue al gestor principal.

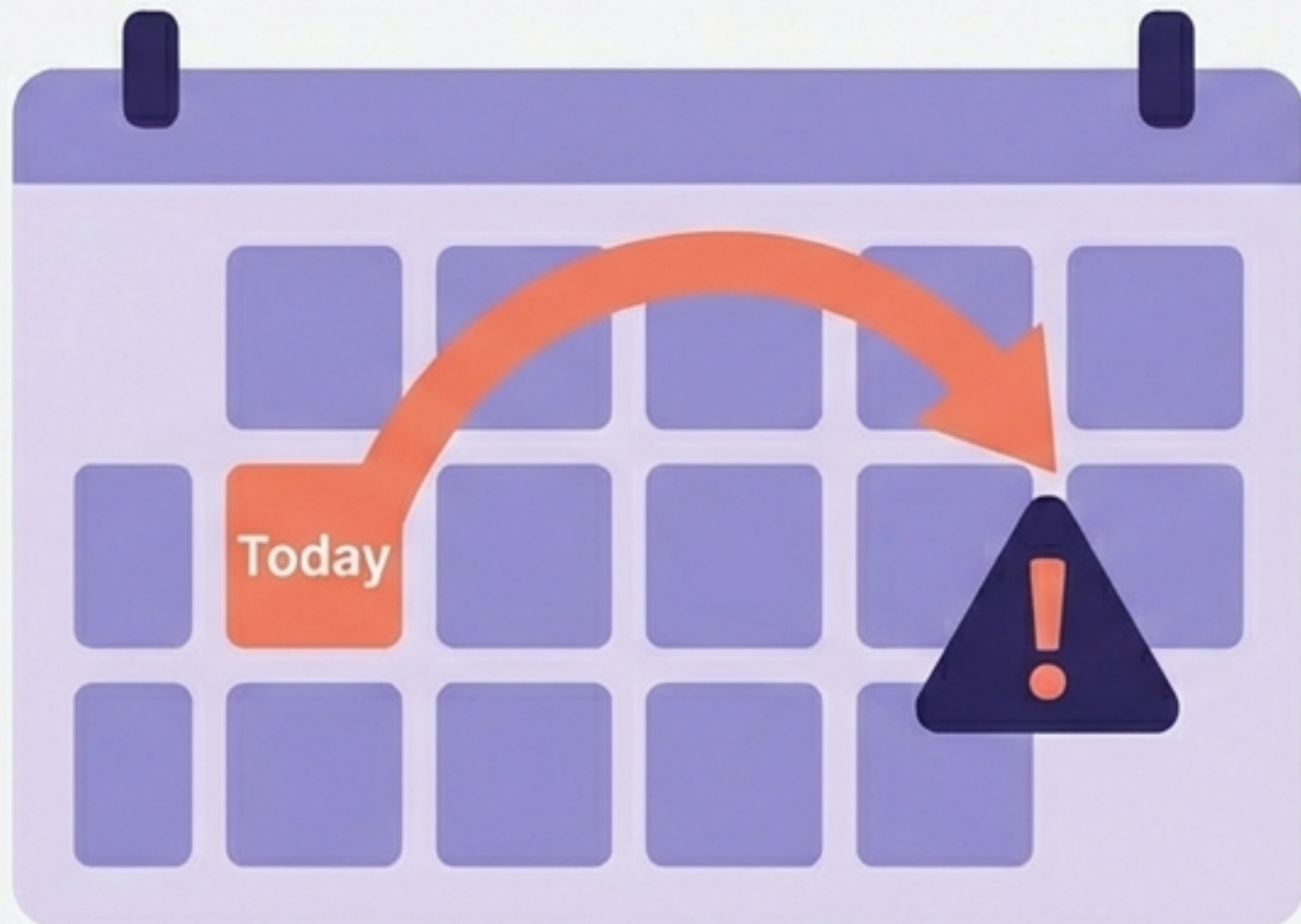


Ejemplo válido: LIB-001

```
if (entrada.matches(Regex("[A-Z]{3}-[0-9]{3}"))) return entrada
```


Control de plazos y vencimientos con `java.time.LocalDate`

Calcular devoluciones y retrasos es exacto y sencillo al instanciar fechas y usar métodos comparativos nativos.



```
// Crear un vencimiento a 14 días
val hoy = LocalDate.now()
val vencimiento = hoy.plusDays(14)

// Comprobar si está vencido
return hoy.isAfter(fechaVencimiento)
```


Una función principal limpia gracias a la delegación de tareas

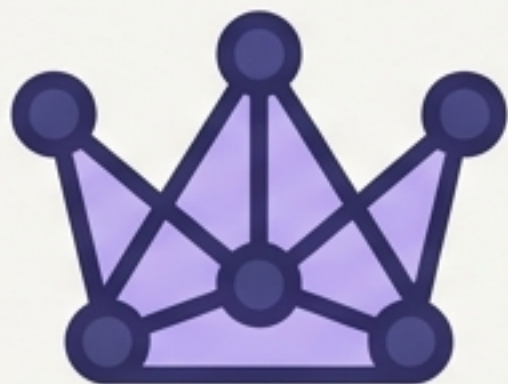
El punto de entrada (main) se limita a ser un enrutador. Todo el peso de la lógica y los datos reside en el Singleton.

```
1. Inventario  
2. Socios  
3. Préstamos  
...
```

```
fun main() {  
    Biblioteca.cargarDatos()  
    while (continuar) {  
        // Interfaz de usuario básica...  
    }  
}
```


Cuatro pilares para una arquitectura robusta en Kotlin

La combinación de estas herramientas transforma un script básico en un sistema profesional.



Singleton (object)

Centraliza el estado y el control sin necesidad de instanciación compleja.



Persistencia

Uso de `java.io.File` para escritura y lectura de memoria a largo plazo.



OOP

Clases abstractas y herencia para un código modular y limpio.



Seguridad

Validaciones estrictas con Regex y gestión delegada de stock.



Tu código es ahora más seguro, centralizado y persistente

Al dominar el patrón object y separar la lógica de negocio de la interfaz, estás listo para escalar tus aplicaciones sin perder el control de tus datos. Construye con confianza.